



Traduction des langages

Amal El Mosslih
Adam Rochdi

Année: 2024/2025

Sommaire

1	Introduction	2
2	Les pointeurs	2
2.1	Modification de l'Ast et Tds	2
2.2	Passe de Gestion d'Identifiants (AstTds)	2
2.3	Passe de Typage (AstType)	3
2.4	Passe de Placement Mémoire (AstPlacement)	3
2.5	Passe de Génération de Code (Code Rat to TAM)	3
3	Variables globales :	3
3.1	Modification de l'ast et tds:	3
3.2	Passe de Gestion d'Identifiants (AstTds):	4
3.3	Passe de Typage (AstType):	4
3.4	Passe Placement Mémoire (AstPlacement):	4
3.5	Passe de Génération de Code (Code Rat to TAM)	5
4	Les variables statiques locales	5
4.1	Modification de l'Ast et Tds	5
4.2	Passe de Gestion d'Identifiants (AstTds)	5
4.3	Passe de Typage (AstType)	5
4.4	Passe Placement Mémoire (AstPlacement):	5
4.5	Passe de Génération de Code (Code Rat to TAM):	6
5	Les paramètres par défauts	6
5.1	Modification de l'AST	6
5.2	Passe de Gestion d'Identifiants (AstTds)	6
5.3	Impact sur les autres passes	6
6	Difficultés rencontrées	7
7	Conclusion	7

1 Introduction

Le projet de programmation fonctionnelle et de traduction des langages (PFTDL) pour l'année 2024/2025 se concentre sur l'extension du compilateur du langage RAT. Ce projet, ancré dans les principes de la programmation fonctionnelle et codé en OCaml, vise à enrichir les capacités du compilateur développé en Travaux Pratiques (TP) en intégrant de nouvelles constructions comme les pointeurs, les variables globales, les variables statiques locales, et les paramètres par défaut. Le rapport détaillera les évolutions apportées au compilateur, les choix de conception, et les jugements de typage associés aux nouvelles fonctionnalités, fournissant ainsi un cadre complet pour la réalisation du projet.

2 Les pointeurs

2.1 Modification de l'Ast et Tds

- L'ajout d'un nouveau cas de type : Pointeur of typ
 - type typ = Bool | Int | Rat | Undefined | **Pointeur of typ**
- L'ajout du nouveau non-terminal : affectable
 - type affectable = | Ident of string | Deref of affectable
- L'ajout de nouveaux cas d'expression :
 - type expression = ... | Affectable of affectable | Null | New of typ
- Modification d'un cas d'instruction :
 - instruction = ... | Affectation of affectable * expression

2.2 Passe de Gestion d'Identifiants (AstTds)

La gestion des identifiants pour les pointeurs dans le projet RAT étendu s'appuie sur la méthode `analyse_tds_affectable`, qui traite les affectables en vérifiant récursivement la validité des pointeurs. Lorsqu'un pointeur est déréférencé (`AstSyntax.Deref`), l'affectable pointé est analysé pour s'assurer qu'il est correctement déclaré.

Pour les identifiants (`AstSyntax.Ident`), la table des symboles est consultée pour vérifier leur déclaration et leur type. Les pointeurs sont transformés en `AstTds.Ident` si leur utilisation est correcte. Toute tentative de modification d'une constante ou une mauvaise utilisation d'un identifiant déclenche une erreur.

Dans `analyse_tds_expression`, les opérations sur pointeurs, comme l'initialisation (`AstSyntax.New`) ou l'accès à une adresse (`AstSyntax.Adresse`), sont vérifiées pour garantir que les identifiants sont valides. Cette gestion assure une utilisation sûre des pointeurs, évitant les erreurs de typage et de déclaration.

2.3 Passe de Typage (AstType)

Lors de la phase de typage, nous avons ajouté un nouveau type, le Pointeur of typ, et adapté les fonctions d'analyse pour gérer les instructions et expressions liées aux pointeurs. Nous avons également intégré de nouveaux jugements de typage dans le système du langage RAT, garantissant ainsi une utilisation cohérente et correcte des pointeurs.

$$\sigma \vdash \text{null} : \text{Pointeur}(\text{Undefined})$$

$$\frac{\sigma \vdash T : \tau}{\sigma \vdash \text{new } T : \text{Pointeur}(\tau)}$$

$$\frac{\sigma \vdash t : \tau}{\sigma \vdash t^* : \text{Pointeur}(\tau)}$$

$$\frac{\sigma \vdash a : \tau}{\sigma \vdash \&a : \text{Pointeur}(\tau)}$$

$$\frac{\sigma \vdash a : \text{Pointeur}(\tau)}{\sigma \vdash *a : \tau}$$

2.4 Passe de Placement Mémoire (AstPlacement)

Lors de la phase de placement en mémoire, la taille allouée à un pointeur est définie à 1. Cette approche reflète le fait que les pointeurs occupent généralement un seul octet en mémoire, cet octet étant utilisé pour stocker l'adresse de référence vers le tas.

2.5 Passe de Génération de Code (Code Rat to TAM)

La gestion des pointeurs dans la génération de code permet un accès et une manipulation efficaces de la mémoire. Lors du déréférencement, la fonction `analyse_code_affectable` analyse récursivement les pointeurs pour obtenir la taille des objets pointés. L'opérateur d'adresse `&` génère le code nécessaire pour charger l'adresse d'une variable ou d'une constante. L'allocation dynamique de mémoire via `new` calcule la taille du type et appelle `Malloc` pour allouer de l'espace. Enfin, les opérations de lecture et de modification des pointeurs sont gérées respectivement par des instructions de chargement (`loadi`) et de stockage (`storei`), garantissant ainsi une gestion correcte et optimisée de la mémoire.

3 Variables globales :

3.1 Modification de l'ast et tds:

J'ai ajouté `InfoGlobal` pour gérer les variables globales :

`type info = ... | InfoGlobal of string * typ * int,`
où `string` est le nom, `typ` le type, et `int` l'adresse mémoire.

Pour l'**AST**, les modifications sont :

- AstSyntax: `type varglobale = Globale of typ * string * expression`
- AstTds: `type varglobale = Globale of typ * Tds.info_ast * expression`

- `AstType`: `type varglobale = Globale of Tds.info_ast * expression`
- `AstPlacement`: `type varglobale= Globale of Tds.info_ast* expression* int`

3.2 Passe de Gestion d'Identifiants (`AstTds`):

La gestion des identifiants pour les variables globales repose sur la méthode `analyse_tds_vglobales`, qui parcourt toutes les variables globales déclarées au début du programme. Cette méthode :

- Vérifie l'absence de double déclaration dans la TDS mère en utilisant `chercherLocalement`.
- Analyse les expressions d'initialisation des variables globales via `analyse_tds_expression`.
- Crée des entrées de type `InfoGlobal` pour chaque variable et les ajoute dans la table des symboles avec `ajouter`.

Dans `analyse_tds_expression` et `analyse_tds_instruction`, les identifiants des variables globales sont recherchés dans la TDS mère à l'aide de `chercherGlobalement`. Et bien sûr, toute tentative de double déclaration ou d'utilisation incorrecte, comme une mauvaise affectation, déclenche une erreur.

3.3 Passe de Typage (`AstType`):

La fonction `analyse_type_vglobales` parcourt toutes les variables globales déclarées et vérifie la compatibilité entre le type déclaré et le type de l'expression d'initialisation. Voici brièvement le fonctionnement de cette fonction:

- Analyse l'expression d'initialisation avec `analyse_type_expression`.
- Vérifie que le type de l'expression est compatible avec le type déclaré à l'aide de `est_compatible`.
- Met à jour le type de la variable globale dans la table des symboles via `modifier_type_variable`.

Si les types sont incompatibles, une exception `TypeInattendu` est levée.

3.4 Passe Placement Mémoire (`AstPlacement`):

La fonction `analyse_placement_vglobales` attribue les adresses mémoire aux variables globales en parcourant la liste des variables déclarées. Elle :

- Calcule la taille mémoire de chaque variable à l'aide de `getTaille`.
- Modifie l'adresse de la variable dans la table des symboles via `modifier_adresse_variable`, en utilisant la base `SB` pour indiquer le segment global.
- Retourne une liste des variables globales avec leur placement en mémoire, représentées sous forme de `AstPlacement.Globale`.

Le déplacement (`depl`) est mis à jour pour chaque variable en fonction de sa taille.

3.5 Passe de Génération de Code (Code Rat to TAM)

La fonction `analyse_code_vglobales` génère le code pour l'initialisation des variables globales. Elle effectue les étapes suivantes :

- Analyse l'expression d'initialisation avec `analyse_code_expression` pour produire le code d'évaluation.
- Récupère la taille mémoire de la variable avec `getTaille`.
- Génère une instruction `store` pour placer la valeur initiale dans le segment global (SB).

4 Les variables statiques locales

4.1 Modification de l'Ast et Tds

- L'ajout d'un nouveau cas d'instruction : Pointeur of typ
 - `type instruction = ... | LocaleStatique of typ * string * expression`

4.2 Passe de Gestion d'Identifiants (AstTds)

Dans la passe de gestion des identifiants, les variables statiques locales sont traitées lors de leur déclaration dans une fonction via `AstSyntax.LocaleStatique`. La fonction `analyse_tds_instruction` vérifie que la variable n'est pas déjà déclarée dans le bloc courant, puis ajoute son information dans la table des symboles locales (`tds`) sous forme d'une `InfoLocal`. Une validation s'assure également que ces variables ne sont pas déclarées dans le programme principal, et une exception (`StatiqueDansMain`) est levée en cas d'erreur.

4.3 Passe de Typage (AstType)

On commence par analyser l'expression e associée à la déclaration locale statique, en appelant `analyse_type_expression`. Ensuite, on compare le type attendu t de la variable locale statique avec le type réel nt de l'expression. Si les types sont compatibles, on modifie le type de la variable à l'aide de `modifier_type_variable`, puis on retourne l'instruction avec l'expression typée. Si les types ne sont pas compatibles, une exception `TypeInattendu` est levée.

4.4 Passe Placement Mémoire (AstPlacement):

Les variables statiques locales sont gérées de manière similaire aux variables globales. Elles sont stockées dans la mémoire statique (segment SB) juste après les variables globales. On utilise une approche en deux phases : d'abord, la fonction `process_static_locals` parcourt toutes les fonctions pour identifier les variables statiques locales et leur allouer un espace dans le segment statique à la suite des variables globales. Ensuite, lors de l'analyse du placement (`analyse_placement_instruction`), quand une variable locale statique est rencontrée (`LocaleStatique`), elle n'est pas analysée pour la deuxième fois car le boolean dans l'info `InfoLocal` est changé à `True`. Cette gestion assure que les variables statiques locales conservent leur valeur entre les appels de fonction, tout en maintenant leur portée locale à la fonction où elles sont déclarées.

Remarque: On a pas réussi à gérer le cas où il n'y a pas de déclaration de variable normale dans le bloc principal avant les appels aux fonctions contenant les variables statiques locales, malgré plusieurs tentatives de débogage.

4.5 Passe de Génération de Code (Code Rat to TAM):

La gestion des variables statiques locales dans la génération de code repose sur l'instruction `LocaleStatique` et un indicateur d'initialisation (`init`) dans la structure `InfoLocal`. Lors de la génération de code, le système vérifie si la variable a déjà été initialisée :

- Si `init` est `false`, le code réserve de l'espace mémoire avec `push`, initialise la variable avec l'expression d'initialisation, et stocke la valeur dans le segment statique **SB** via `store`. - Si `init` est `true`, aucun code n'est généré.

Ce mécanisme assure que chaque variable statique locale est initialisée une seule fois, même si la fonction contenant la variable est appelée plusieurs fois.

5 Les paramètres par défauts

5.1 Modification de l'AST

Une seule modification a été apportée à l'AST :

- Ajout du type `default` : `Default of expression`
- Ajout du type fonction : `Fonction of typ * string * (typ * string * default option) list * bloc`

5.2 Passe de Gestion d'Identifiants (AstTds)

La gestion des paramètres par défaut a été implémentée à travers une table dédiée `Tdp` qui stocke les valeurs par défaut des paramètres pour chaque fonction. Cette table est utilisée lors de l'analyse des appels de fonction pour compléter automatiquement les arguments manquants.

Le processus se déroule en deux phases principales:

1. **Déclaration des paramètres:** Lors de l'analyse d'une déclaration de fonction, nous vérifions que les paramètres par défaut respectent la règle suivante: une fois qu'un paramètre par défaut est déclaré, tous les paramètres suivants doivent également avoir une valeur par défaut. Cette vérification est effectuée par la fonction `verifier_params_default`.
2. **Appels de fonction:** À chaque appel de fonction, la fonction `completer_arguments` compare le nombre d'arguments fournis avec le nombre total de paramètres. Si des arguments sont manquants, elle les complète automatiquement avec les valeurs par défaut stockées dans la `table_param`.

L'interface `tdp.mli` définit les opérations essentielles pour cette gestion, notamment `ajouter_params` pour l'enregistrement des paramètres et `completer_arguments` pour leur utilisation lors des appels.

5.3 Impact sur les autres passes

Dans les passes suivantes (typage, placement mémoire et génération de code TAM), aucune modification n'a été nécessaire pour gérer les paramètres par défaut. En effet, une fois la passe de gestion des identificateurs effectuée, les paramètres par défaut sont traités exactement comme des paramètres normaux, car la complétion des arguments manquants est entièrement résolue lors de la première passe.

6 Difficultés rencontrées

Lors de l'implémentation **des paramètres par défaut**. La première approche intuitive consistait à définir les paramètres comme des expressions optionnelles, en utilisant le type **expression option**. Cependant, cette solution a rapidement révélé une limitation majeure : elle créait un cycle de dépendances entre les modules.

En effet, l'analyse d'une expression nécessite l'accès à la table des paramètres (pour gérer les appels de fonctions avec paramètres par défaut), mais la table des paramètres aurait elle-même contenu des expressions, créant ainsi **une dépendance circulaire**. Cette situation nous a conduits à adopter une approche alternative avec un type **defaut** distinct et une table dédiée **table_param**, permettant de briser ce cycle tout en maintenant la fonctionnalité souhaitée.

La gestion en mémoire des variables statiques locales a été tout aussi problématique. En effet, On a d'abord essayé leurs adresses dans la pile et elles dans le tas, mais on a pas réussi à l'implémenter. Ensuite, on a choisi de les gérer dans la SB juste après les variables globales, ce qui marche sauf pour le cas où il n'y a pas une instruction dans le bloc principal avant le premier appel de fonction.

7 Conclusion

Ce projet de compilateur nous a permis d'enrichir le langage RAT avec de nouvelles fonctionnalités significatives : les variables statiques locales, les variables globales, les pointeurs et les paramètres par défaut. Cette expérience a été enrichissante car elle a mêlé application pratique des concepts de compilation et exploration des principes fondamentaux de la traduction de langages. Plusieurs améliorations pourraient être envisagées, comme l'optimisation de l'espace mémoire ou l'ajout d'une gestion automatique de la mémoire.